41026 Advanced Software Development

Prediction Market Intelligence Hub

Release 2: Multi-Agent Systems, Testing & Cloud Deployment

Assessment 3: Release 2 (30%)

Technical Report

Team Name: Group 29
Workshop Class / Tutor: Grace Billiris

Student #	Full Name	Student ID	GitHub Username	Assigned Feature
Student 1	Alex Choi	26255859	AlexChoi2004	Market Watchlist & Discovery
Student 2	Christopher Kumarelil Cherian	25491198	christopherkumarelilcherian-bit	Portfolio & Position Tracker
Student 3	Syed Humza Hussain	25474457	SHumzaHussain – Silv3r-exe	Market Research Notes & News Feed
Student 4	Erfanur Rahman	24951459	Erfanur1	AI Market Analyst Assistant

GitHub Repository URL: https://github.com/Erfanur1/ADVSD
Submission Date: 25 October 2026, 11:59 PM AEST
Report file name: *group-29.pdf*

1. Project Overview

The Prediction Market Intelligence Hub is an Agentic AI microservices application that lets users browse, search and track prediction markets, and ask AI-grounded questions about live market data sourced from Polymarket and Kalshi. Release 0 delivered the core microservices (frontend, backend/API and database per student) and a containerised Docker Compose deployment with a shared local AI-Mode service. Release 1 extended the application with a shared Model Context Protocol (MCP) server, a shared Retrieval-Augmented Generation (RAG) server, live market data in place of seeded data, and an extended Plan → Act → Observe → Adapt agentic loop capable of validating both the MCP and RAG servers.

Release 2 completes the Agentic AI application by adding a shared, non-containerised Multi-Agent Server (Planner, Worker and Reviewer agents, with a human-in-the-loop review step), extending the shared agentic loop with three further review modes (Multi-Agent Workflow Review, Testing Report Review and Cloud Deployment Report Review), introducing local pre-commit security scanning and post-commit CI/CD endpoint testing for every student's service, and deploying the fully integrated application to a public cloud platform (Microsoft Azure or AWS), with the AI-Mode, MCP, RAG and Multi-Agent services disabled by default in that cloud deployment.

1.1 Team Members and Feature Allocation

Student #	Full Name	Student ID	GitHub Username	Assigned Feature
Student 1	Alex Choi	26255859	AlexChoi2004	Market Watchlist & Discovery
Student 2	Christopher Kumarelil Cherian	25491198	christopherkumarelilcherian-bit	Portfolio & Position Tracker
Student 3	Syed Humza Hussain	25474457	SHumzaHussain – Silv3r-exe	Market Research Notes & News Feed
Student 4	Erfanur Rahman	24951459	Erfanur1	AI Market Analyst Assistant

1.2 How Release 2 Extends Release 1

Release 2 keeps the Release 1 application unchanged in principle: AI-Mode, the MCP server and the RAG server remain shared, non-containerised local services, each student's frontend/backend/database microservices stay containerised through Docker Compose, and live market data continues to come from Polymarket and Kalshi. On top of that, Release 2 adds a fourth shared non-containerised service, the Multi-Agent Server, which coordinates a Planner, Worker and Reviewer agent and includes a human review step before a result is returned to the user. The shared agentic loop is extended with three new review modes (Multi-Agent Workflow Review, Testing Report Review and Cloud Deployment Report Review) alongside its existing MCP and RAG validation modes. Each student adds local pre-commit security scanning and two post-commit CI/CD endpoint tests to their own service, and the team deploys the fully integrated application to a public cloud platform, with AI-Mode, MCP, RAG and the Multi-Agent Server disabled by default in that cloud deployment.

1.3 Individual Feature Updates and Release 2 Responsibilities

Student	Individual Release 2 Update	Shared Group Responsibility (claimed)
Student 1 (Alex Choi)	Market Watchlist & Discovery integrated with the shared Multi-Agent Server (POST /multi-agent/validate and POST /multi-agent/review, with graceful fallback and a Plan/Act/Observe/Adapt trace). Local pre-commit security scanning (Ruff, detect-secrets, pip-audit) plus a pytest hook. 14 new endpoint tests for POST /watchlist and GET /markets, and a post-commit AI-assisted unit testing job in student-1.yml. MULTI_AGENT_URL environment variable added.	*Claimed all four shared tasks: (1) shared Multi-Agent Server, (2) shared pre-commit configuration, (3) Azure deployment IaC and scripts (Bicep), (4) cloud-deployment.yml workflow.*
Student 2 (Christopher Kumarelil Cherian)	Portfolio Tracker integrated with the Multi-Agent Server through a new review_portfolio task: POST /multi-agent/validate, POST /multi-agent/review and a “Multi-Agent Portfolio Review” panel with the human decision form. Pre-commit: 2 Ruff S110 findings fixed, S104 documented, 8 dependency CVEs fixed by upgrading. 16 tests covering GET /positions, GET /trade-history and the Multi-Agent routes, run by student-2.yml with a container smoke test. MULTI_AGENT_URL added.	Added the review_portfolio task type (Planner, Worker, Reviewer) and the --portfolio-id terminal option to the shared Multi-Agent Server
Student 3 (Syed Humza Hussain)	Market Research Notes and News Feed integrated with the shared Multi-Agent Server (POST/ Multi-agent/validate and POST/Multi-agent/review with a graceful fallback and a PLAN/ACT/OBSERVE/ADAPT trace). A new "Fact check a note (Multi-agent)" panel checks a research notes claimed probability against a live market through the validate_market_action task and shows the Reviewers recommendation with Approve/ Partial/ Reject buttons for the human decision. Local pre-commit security scanning (Ruff, Detect-secrets, pip-audit) plus a pytest hook. 20 new endpoint tests (26 in total) for GET/news Post/notes and the multi agent routes run by student-3.yml.	Updated the shared configuration files for the student 3 service: added the pytest-student-3 hook to the shared .pre-commit-config.yml added the MULTI-AGENT-URL to teh student 3 backend service in the docker-compose.yml and added the ignore entry to .gitignore
Student 4 (Erfanur Rahman)	AI Market Analyst Assistant integrated with the shared Multi-Agent Server through a new review_mispricing_analysis task (Planner plans a re-check of a Polymarket/Kalshi pair through the MCP compare_markets tool; the Worker runs it; the Reviewer flags last-trade bases and gaps over 20 points). Adds POST /multi-agent/validate, POST /multi-agent/review and a "Run Multi-Agent Review" button in the compare-pair flow, with the human decision form. Also adds MULTI_AGENT_URL and the missing pytest-student-4 pre-commit hook. student-4.yml runs the feature, shared-data-layer and compare_markets tests, then smoke-tests the container with MCP, RAG and Multi-Agent disabled.	Took over and got the Azure cloud deployment (shared tasks 3 and 4) working end to end: AI-Mode disabled by default in the cloud, OIDC managed-identity login for cloud-deployment.yml, a more reliable verify-CI gate, deploy.sh fixes (deterministic ACR name, docker build+push, a retried second pass), Azure Files removed because Express environments don't support it, and the Container Apps API version bumped.

2. Final Repository Structure

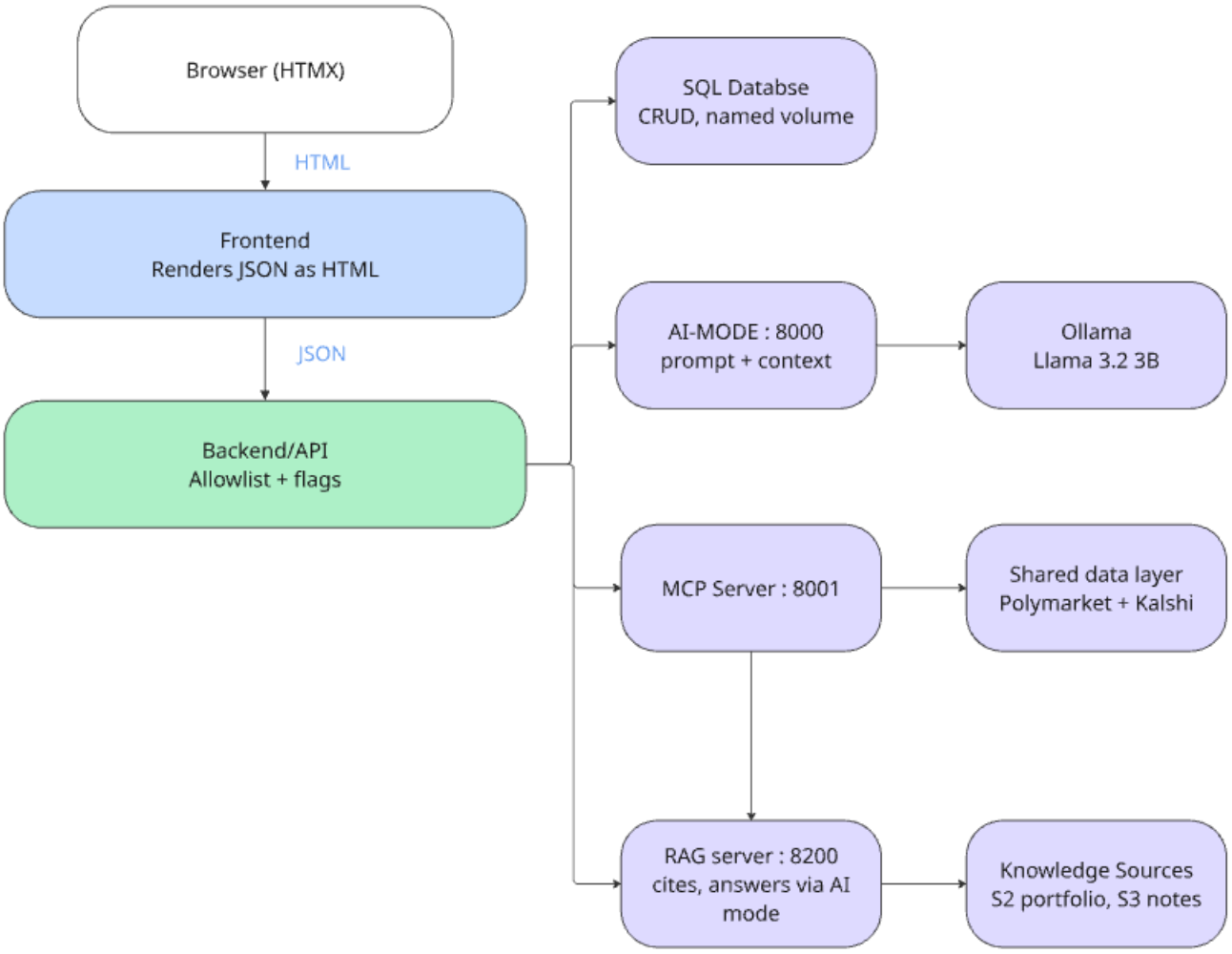
This section shows the final shared GitHub repository layout, carrying over the Release 0 and Release 1 structure and adding the Release 2 artefacts. Items marked [NEW] do not exist yet and are the shared Release 2 build tasks; everything else is retained unchanged from Release 1.

```
ADVSD/
├── .pre-commit-config.yaml          [NEW – shared, Section 5]
├── .secrets.baseline                [NEW – shared, detect-secrets]
├── docker-compose.yml               (unchanged: student services only)
├── .github/workflows/
│   ├── student-1.yml                (extended: +2 endpoint tests)
│   ├── student-2.yml                (extended: +2 endpoint tests)
│   ├── student-3.yml                (extended: +2 endpoint tests)
│   ├── student-4.yml                (extended: +2 endpoint tests)
│   └── cloud-deployment.yml         [NEW – shared, Section 4]
├── deploy/                          [NEW – shared, cloud IaC/scripts]
│   └── [azure|aws]/...
├── ai-services/
│   ├── ai-mode/                     (retained, non-containerised)
│   ├── mcp-server/                  (retained, non-containerised)
│   ├── rag-server/                  (retained, non-containerised)
│   ├── multi-agent-server/          [NEW – shared, Section 10]
│   │   ├── app.py
│   │   ├── planner.py
│   │   ├── worker.py
│   │   ├── reviewer.py
│   │   └── human_review.py
│   ├── agentic-loop/
│   │   └── loop.py                  (extended: +3 review modes)
│   └── shared/
│       └── market_data.py           (retained from Release 1)
├── student-1/  (frontend, backend, database, tests)
├── student-2/  (frontend, backend, database, tests)
├── student-3/  (frontend, backend, database, tests)
└── student-4/  (frontend, backend, database, tests)
```

3. Release 2 Design Flow

The design flow below shows how a request moves through the integrated application, including the Release 1 AI flows (AI-Mode, MCP, RAG) that are retained unchanged, and the new Release 2 Multi-Agent flow. The backend is the only gateway to the shared AI services. Every feature keeps one rule from Release 1: the browser talks HTML to its frontend, and everything behind the frontend exchanges JSON, with the backend/API as the only gateway to the database and the shared services.

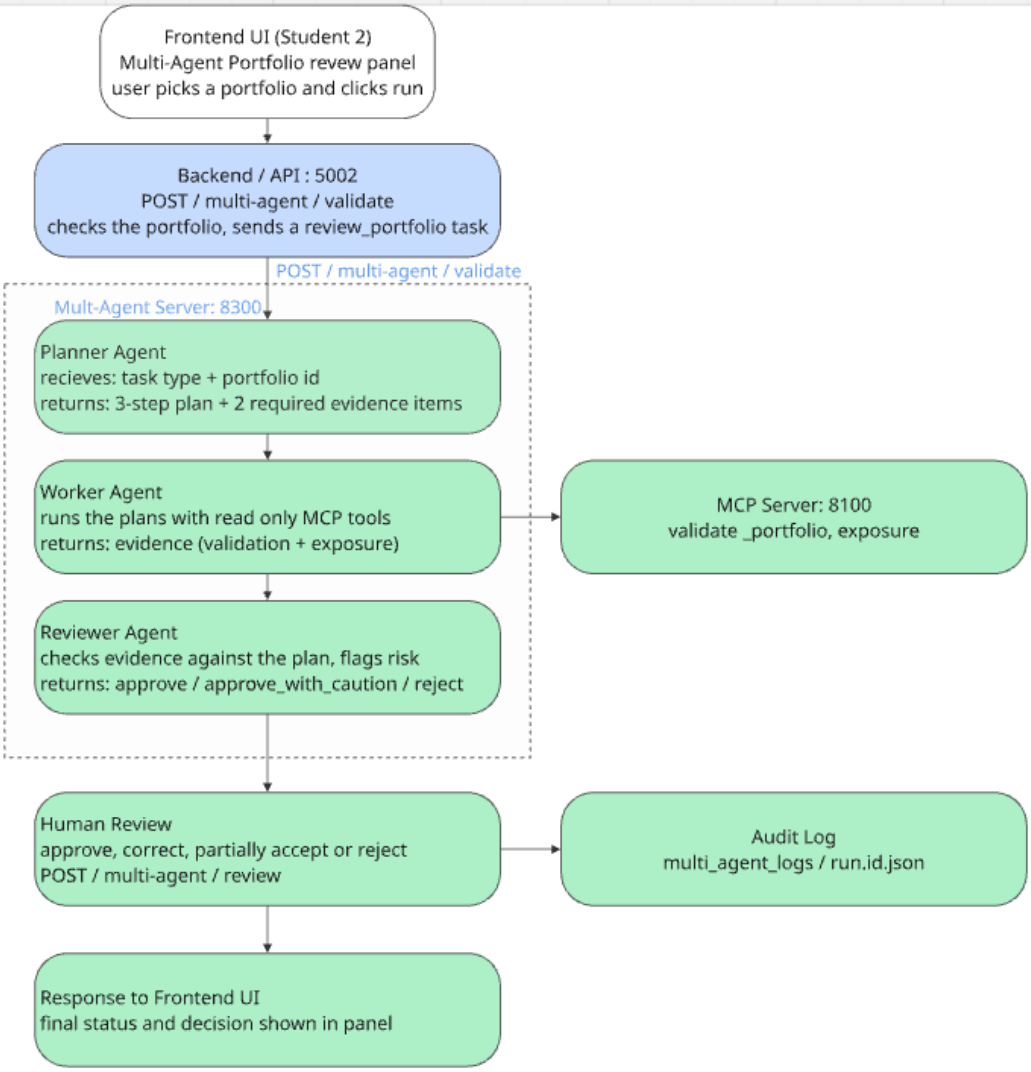
3.1 Core and AI Request Flow (retained from Release 0/1)



CRUD goes from the frontend to the backend to SQLite. AI-Mode passes a prompt to Ollama; MCP runs registered tools over the shared live data layer; RAG retrieves context from several features and live markets, then asks AI-Mode for a grounded answer with citations. All four student features follow this shape.

3.2 Multi-Agent Workflow Flow (new in Release 2)

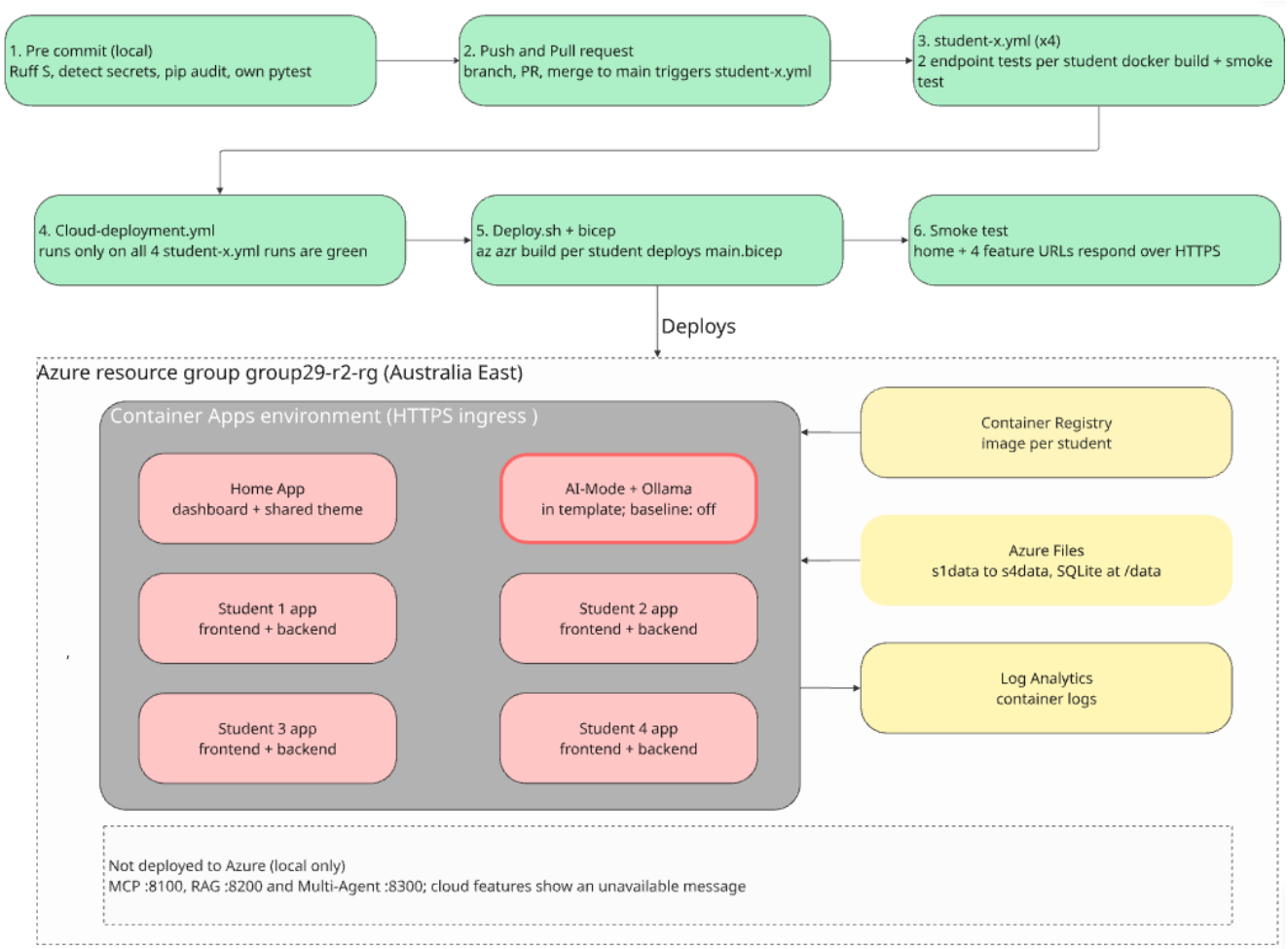
A person makes the final call after the Planner, Worker and Reviewer agent



Nothing is applied until a person decides: the run waits as pending_human_review, and the decision is written to the audit log. Every student's backend calls this same server and differs only in the task type it sends (Student 1: validate_market_action; Student 2: review_portfolio).

4. Cloud Deployment and CI/CD Architecture

Platform selected: Microsoft Azure (Azure Container Apps, Azure Container Registry, Azure Files for SQLite persistence, Log Analytics), provisioned with Bicep. The group deploys Microsoft Azure Container Apps, provisioned with Bicep, and a deployment only starts after every student's CI workflow on main is green.



Each student frontend and backend run as one Container App behind HTTPS ingress, with its SQLite file on an Azure Files share, so data survives restarts. Images are built in Azure Container Registry by `deploy.sh`, and logs go to Log Analytics. AI-Mode and Ollama are in the current template (dashed amber box): for the required baseline they must be off by default or claimed as bonus B1 (Section 13).

5. Pre-Commit Security Testing

Pre-commit security scans run locally whenever code is committed, before it reaches GitHub. The team uses a shared pre-commit configuration covering Ruff's security rules, detect-secrets, and pip-audit (or approved equivalents).

5.1 Shared Scan Configuration

The repository root holds a single shared .pre-commit-config.yaml covering all four students' backend/API code, with three hooks: Ruff's security rule set (--select S, flagging issues such as use of eval/exec, hardcoded bind-all addresses, and insecure subprocess calls); detect-secrets, checked against a committed .secrets.baseline so new commits are diffed against a known-clean snapshot rather than re-flagging accepted false positives; and pip-audit, which checks each service's installed dependencies against the public vulnerability database. Each student installs the hooks once per clone with pip install pre-commit followed by pre-commit install, after which all three scans run automatically on every git commit and block the commit on a new finding.

Hook	What it checks	Notes

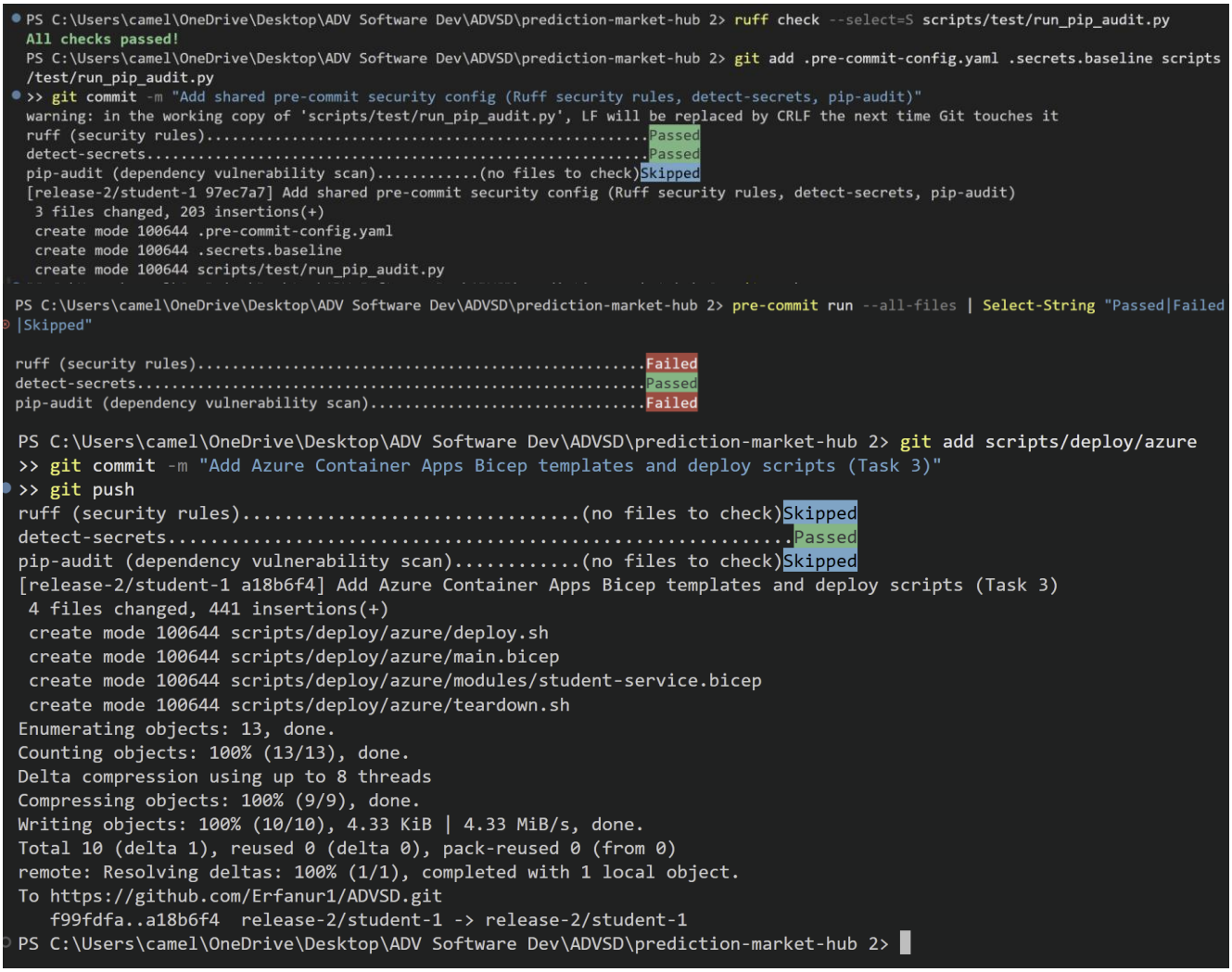
ruff	Security rules: eval/exec, bind-all addresses (S104), swallowed exceptions (S110), subprocess use	Test files excluded (S101 assert noise)
detect-secrets (v1.5.0)	New keys, tokens or passwords	Diffed against the committed .secrets.baseline
pip-audit	`Known CVEs in each changed requirements.txt`	Runs through scripts/test/run_pip_audit.py, which works on Windows, macOS and Linux
pytest (per student)	That student's endpoint tests	Runs only when files under that student's folder change

5.2 Per-Student Results

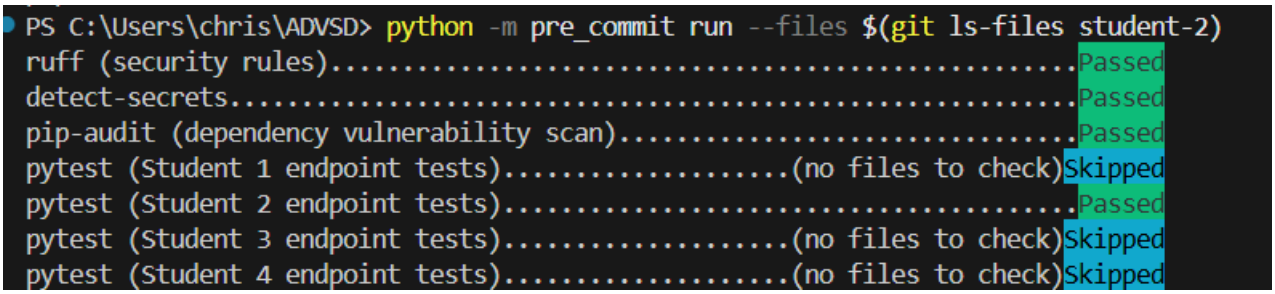
Student	Ruff (security rules)	detect-secrets	pip-audit	Outcome
Student 1 – Alex Choi	1x S104 (binding to 0.0.0.0, required for Docker), documented and suppressed with justified noqa; 4x S110, resolved	Passed, no secrets detected	10 CVEs/4 packages (backend); 8 CVEs/3 packages (frontend), documented, deferred	All findings resolved or documented as accepted exceptions
Student 2 – Christopher K. Cherian	2× S110 (`except: pass` in the add and close position routes) fixed by logging the error; 2× S104 documented with the agreed noqa	Passed	8 findings in 3 packages per service (Flask 3.0.3, requests 2.32.3, pytest 8.3.3), fixed by upgrading to 3.1.3, 2.33.0 and 9.0.3; re-scan clean	All clean
Student 3 – Syed Humza Hussain	4 x S110 (silent except: pass in the frontend note routes), resolved by logging the error 2xS104 (binding to 0.0.0.0 in the backend and front end required for Docker) documented with justification	Passed no secrets detected	10 vulnerabilities in 4 packages (backend) 8 in 3 packages (frontend)Flask 3.0.3 requests 2.32.3 pytest 8.3.3 pythong-dotenv 1.0.1. Fixed by upgradeing to 3.1.3, 2.33.0, 9.0.03 and 1.22	All Clean

Student	Ruff (security rules)	detect-secrets	pip-audit	Outcome
Student 4 – Erfanur Rahman	2× S104 (backend and frontend 0.0.0.0 bind) not yet suppressed; 3× S110 (try/except/pass in frontend/app.py lines 257, 267, 276)	Passed, no secrets detected	10 findings in 4 packages (backend) and 8 in 3 packages (frontend): Flask 3.0.3, requests 2.32.3, pytest 8.3.3, python-dotenv 1.0.1; fixed versions 3.1.3, 2.33.0, 9.0.3, 1.2.2	Outstanding: see Section 15 *(update to "All clean" if fixed before submission)*

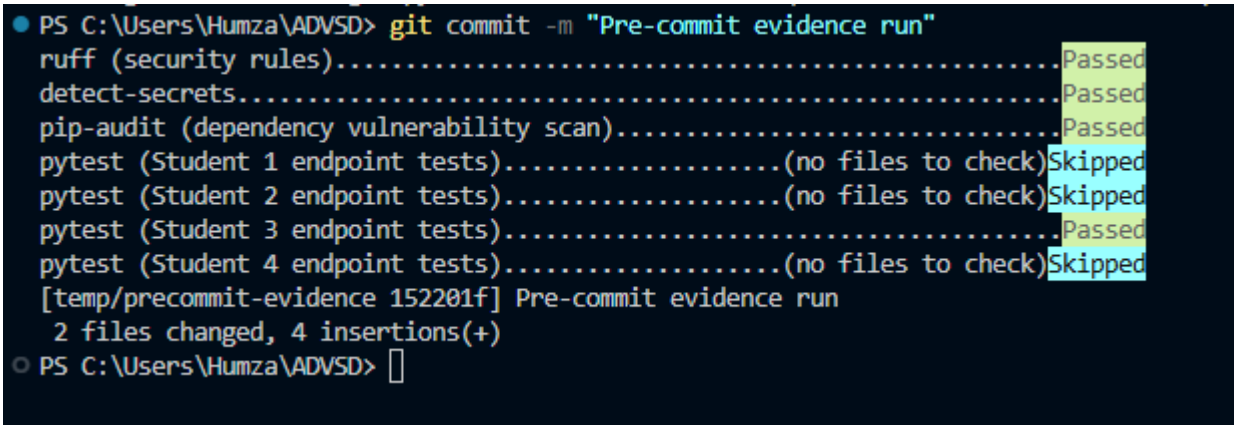
Student 1:



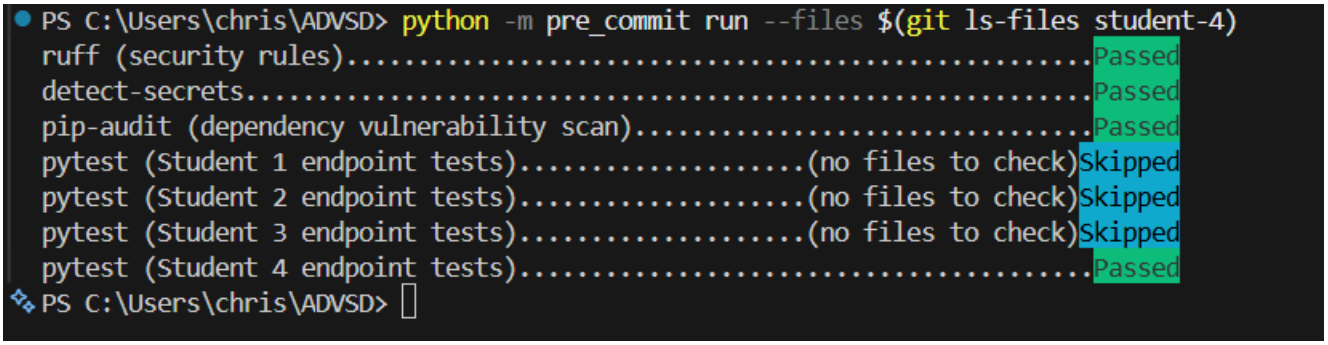
Student 2:



Student 3:



Student 4:



6. Post-Commit CI/CD Testing

After code is pushed, each student's assigned student-x.yml GitHub Actions workflow runs automated tests against two endpoint functions of that student's backend/API service. Student 1 and Student 2's endpoints below are confirmed from their existing Release 0/1 routes; Students 3 and 4 should confirm the two endpoints their tests will cover.

6.1 Endpoint Test Summary

Student	Endpoint 1	Endpoint 2	Workflow Run Status
Student 1 – Alex Choi	POST /watchlist	GET /markets	student-1-ci #29 (main): passed. build-and-validate (19 tests, pytest pre-commit validation) and post-commit-ai-unit-testing both successful. https://github.com/Erfanur1/ADVSD/actions/workflows/student-1.yml
Student 2 – Christopher K. Cherian	GET /positions	GET /trade-history	student-2-ci on PR #21 and on `main` after merge `f9187f2`. https://github.com/Erfanur1/ADVSD/actions/workflows/student-2.yml
Student 3 – Syed Humza Hussain	GET /news	POST /notes	student-3-ci #22 (main): passed, after merging PR #22 (da7c989). build-and-validate (26 tests, pytest pre-commit validation) and post-commit-ai-unit-testing both successful.

Student	Endpoint 1	Endpoint 2	Workflow Run Status
			https://github.com/Erfanur1/ADVSD/actions/workflows/student-3.yml
Student 4 – Erfanur Rahman	POST /mcp/execute	POST /mispricing/save	student-4-ci on PR #34 and on main after merge 0dd9a27: passed. https://github.com/Erfanur1/ADVSD/actions/workflows/student-4.yml

Student 1:

- POST /watchlist (4 tests): a valid request returns 201 with the new item's id, and the item then appears in GET /watchlist with the submitted market_id, note and priority. When note and priority are omitted, they default to "" and 0. Edge cases: a missing market_id returns 400 with an error message, and an unknown market_id returns 404.

- GET /markets (4 tests): returns 200 with a JSON list of at least 10 markets sorted by volume in descending order. The category filter returns only markets in that category, and the q search finds a known market title. Edge case: a search with no match returns 200 with an empty list.

- Plus 6 tests for the Multi-Agent routes (/multi-agent/validate and /multi-agent/review), including the fallback when the Multi-Agent Server is unreachable.

Student 2:

- GET /positions (5 tests): returns 200 with at least 10 seeded rows, each with a portfolio name; every row has the expected fields, a YES/NO side, a price between 0 and 1 and a positive integer size; ?portfolio_id=3 returns only that portfolio; an unknown portfolio returns 200 with an empty list; a newly opened position appears in the list.

- GET /trade-history (4 tests): returns 200 with at least 10 BUY/SELL trades; rows are newest first; each row carries the joined market_ticker (the seeded SELL is POL-TRUMP-2026, 20 shares at 0.58); closing a position removes its trades (cascade delete).

- Multi-Agent routes (7 tests, shared server mocked): disabled returns 503; a missing or non-integer portfolio_id returns 400; an unknown portfolio returns 404; a valid request sends a review_portfolio task and returns a pending run with a four-stage trace; an unreachable server returns 502; an invalid human decision returns 400; a valid decision is passed through.

Student 3:

- GET /news (6 tests): returns 200 with at least 10 articles, ordered newest first, each with its source name. The category filter returns only that category and the headline search finds a known headline. The two filters combine. Edge case a search with no match returns 200 with an empty list.

- POST /notes (5 tests):a valid request returns 201 and the note then appears in GET /notes with the submitted article_id, title and tags and its joined headline. Optional fields default to empty strings. Edge cases: a missing article_id returns 400, and an unknown article returns 404. Adding a note does not change other notes.

- Plus 9 tests for the mutli-agent routes with the shared server mocked: a missing note_id or market_id returns 400, an unknown note returns 404, an out-of-range or non-numeric claimed probability returns 400, and an unreachable server returns a fallback with a four stage trace. A valid request sends a validate_market_action task that returns a run pending human review. The review route returns 400

without a run_id and decision, 503 if the server is unreachable and passes a valid decision through. In total: the 6 existing tests plus 20 new ones

Student 4:

- **POST /mcp/execute (5 tests):** a compare_markets call returns 200 with the comparison result and a four-stage Plan/Act/Observe/Adapt trace, sending exactly the tool and parameters given; tools outside the allow-list (e.g. validate_portfolio, drop_tables) are rejected with 400; an MCP error such as "market not found" is passed through as 404; an unreachable MCP server returns 502 "unavailable"; disabled in CI returns 503 "disabled".

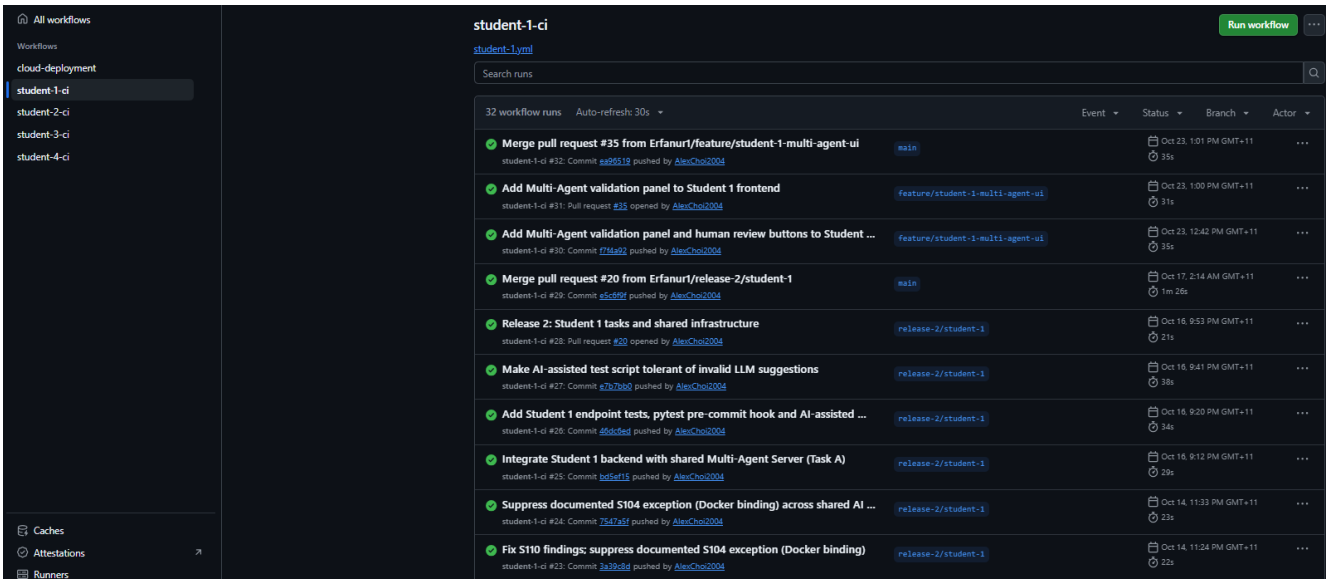
- **POST /mispricing/save (2 tests):** saving a pair upserts the market (title, probability, close date) and stores an analysis whose summary includes the gap ("10.0 pts apart") and the Kalshi ticker; saving the same pair again reuses the market row and adds a new analysis. Disabled in CI returns 503.

- **Multi-Agent routes (6 tests, shared server mocked):** disabled returns 503; both polymarket_id and kalshi_ticker are required; a valid request sends a review_mispricing_analysis task and returns a pending run; an unreachable server returns 502; an invalid human decision returns 400; a valid decision is passed through.

6.2 GitHub Actions Workflow Evidence

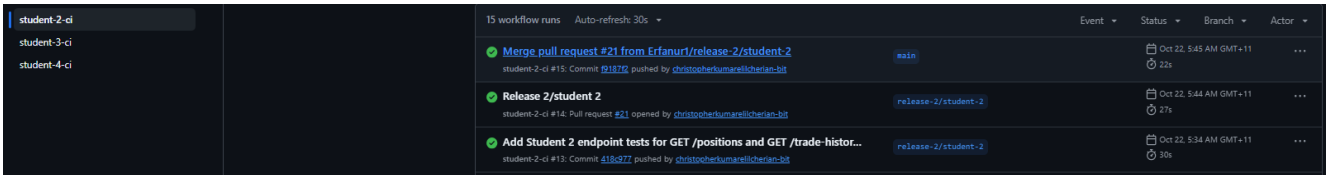
Screenshots of each student's workflow run history, in chronological order, as in the Release 0 and Release 1 reports.

Student-1-ci:



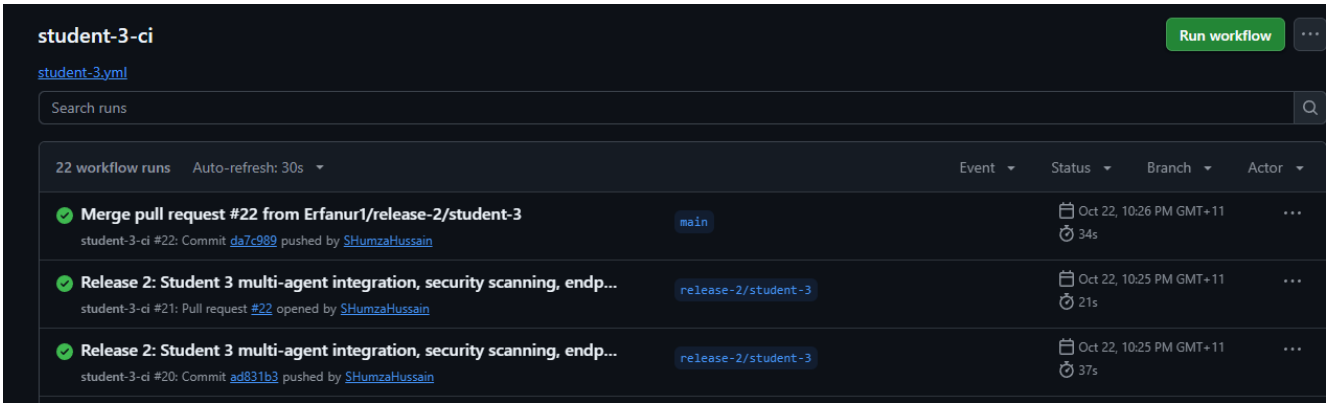
1. **#23** Fixed the S110 findings and documented the S104 Docker-binding exception (3a39c8d).
2. **#24** Applied the S104 suppression across the shared AI services and backend entrypoint (7547a5f).
3. **#25** Integrated the Student 1 backend with the shared Multi-Agent Server, Task A (bd5ef15).
4. **#26** Added the endpoint tests for POST /watchlist and GET /markets, the pytest pre-commit hook and the AI-assisted test stage, Task C (46dc6ed).
5. **#27** Made the AI-assisted test script tolerant of invalid LLM suggestions (e7b7bb0).
6. **#28** Opened pull request #20 (Release 2: Student 1 tasks and shared infrastructure); CI passed on the pull request.
7. **#29** Merged PR #20 into `main` (e5c6f9f); CI passed on `main`.
8. **#30** Added the Multi-Agent validation panel and human review buttons to the Student 1 frontend (f7f4a92).
9. **#31** Opened pull request #35; CI passed on the pull request.
10. **#32** Merged PR #35 into `main` (ea96519); CI passed on `main`.

Student-2-ci:



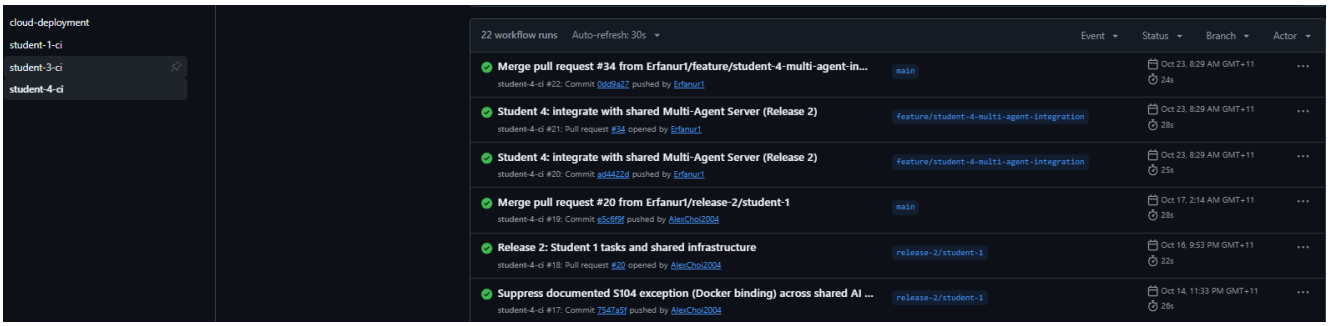
1. **#13** Added the Student 2 endpoint tests for GET /positions and GET /trade-history, plus the pytest pre-commit hook, Task C (418c977). This run covers 16 tests in a separate “Post-commit endpoint tests” step, uploads the JUnit results as an artifact, and runs a container smoke test with Multi-Agent, MCP and RAG disabled.
2. **#14** Opened pull request #21 (Release 2/student 2), which includes the Multi-Agent integration (review_portfolio task, backend routes, UI panel, MULTI_AGENT_URL); CI passed on the pull request.
3. **#15** Merged PR #21 into `main` (f9187f2); CI passed on `main`.

Student-3-ci:



1. Pushed Release 2 work to release-2/student-3 (ad831b3): Multi-Agent integration, endpoint tests, pre-commit fixes and the post-commit AI testing job. Build-and-validate and post-commit-ai-unit-testing both passed
2. Opened pull request #22: build-and-validate passed on the pull request (the post-commit job is skipped on pull requests by design)
3. Merged pull request #22 into main (da7c989). CI passed on main

Student-4-ci:



1. **#17** The shared S104 suppression in the AI services (7547a5f) touched the shared MCP and data-layer code, so Student 4’s CI re-ran as a regression check and passed.
2. **#18** Pull request #20 (Student 1 tasks and shared infrastructure) re-ran Student 4’s tests against the shared changes; passed.
3. **#19** Merged PR #20 into `main` (e5c6f9f); Student 4 still passed after the shared changes.
4. **#20** Integrated Student 4 with the shared Multi-Agent Server (ad4422d): review_mispricing_analysis task, backend routes, “Run Multi-Agent Review” button, MULTI_AGENT_URL, pytest-student-4 hook.
5. **#21** Opened pull request #34; CI passed on the pull request.
6. **#22** Merged PR #34 into `main` (0dd9a27); CI passed on `main`, including the container smoke test confirming MCP, RAG and Multi-Agent report disabled.

7. Testing Review (Agentic-Loop Testing Review Mode)

The shared agentic loop's new Testing Review mode reads the pre-commit security testing results and the post-commit CI/CD endpoint test results from Sections 5 and 6, and produces a review with recommendations.

```
PS C:\Users\chris\ADVSD> python ai-services/agentic-loop/loop.py --mode testing-review --skip-audit
=== Agentic Loop: Testing Report Review Mode ===
  [Plan] Review pre-commit security evidence (Ruff S rules, detect-secrets, pip-audit) and post-commit test evidence (pytest + latest student-x.yml run on main)
 for student-1, student-2, student-3, student-4.
  [Act] Running the scans and test suites locally and reading CI status from GitHub Actions.
    ... scanning student-1
    ... scanning student-2
    ... scanning student-3
    ... scanning student-4
  [Observe] student-1: Low risk; student-2: Low risk; student-3: Low risk; student-4: Low risk
  [Adapt] Overall risk Low. 1 recommendation(s) produced.
    - All scans clean and all tests passing; no action needed.

[saved validation log: docs\release-2\review-logs\testing-review_20261028_235845.json]
[saved review report:  docs\release-2\review-logs\testing-review_20261028_235845.md]
PS C:\Users\chris\ADVSD>
```

Testing Report Review

Overall risk: Low · **Generated:** 2026-10-28 23:58

Stage	Detail
Plan	Review pre-commit security evidence (Ruff S rules, detect-secrets, pip-audit) and post-commit test evidence (pytest + latest student-x.yml run on main) for student-1, student-2, student-3, student-4.
Act	Running the scans and test suites locally and reading CI status from GitHub Actions.
Observe	student-1: Low risk; student-2: Low risk; student-3: Low risk; student-4: Low risk
Adapt	Overall risk Low. 1 recommendation(s) produced.

Evidence by student

Student	Ruff (S rules)	detect-secrets	pip-audit	Local pytest	CI on main	Risk
student-1	clean	clean	skipped	19/19 passed	success (#32, ea96519)	Low
student-2	clean	clean	skipped	30/30 passed	success (#15, f9187f2)	Low
student-3	clean	clean	skipped	26/26 passed	success (#22, da7c989)	Low
student-4	clean	clean	skipped	26/26 passed	success (#22, 0dd9a27)	Low

Risk notes

- **student-1:** no issues
- **student-2:** no issues
- **student-3:** no issues
- **student-4:** no issues

Recommendations

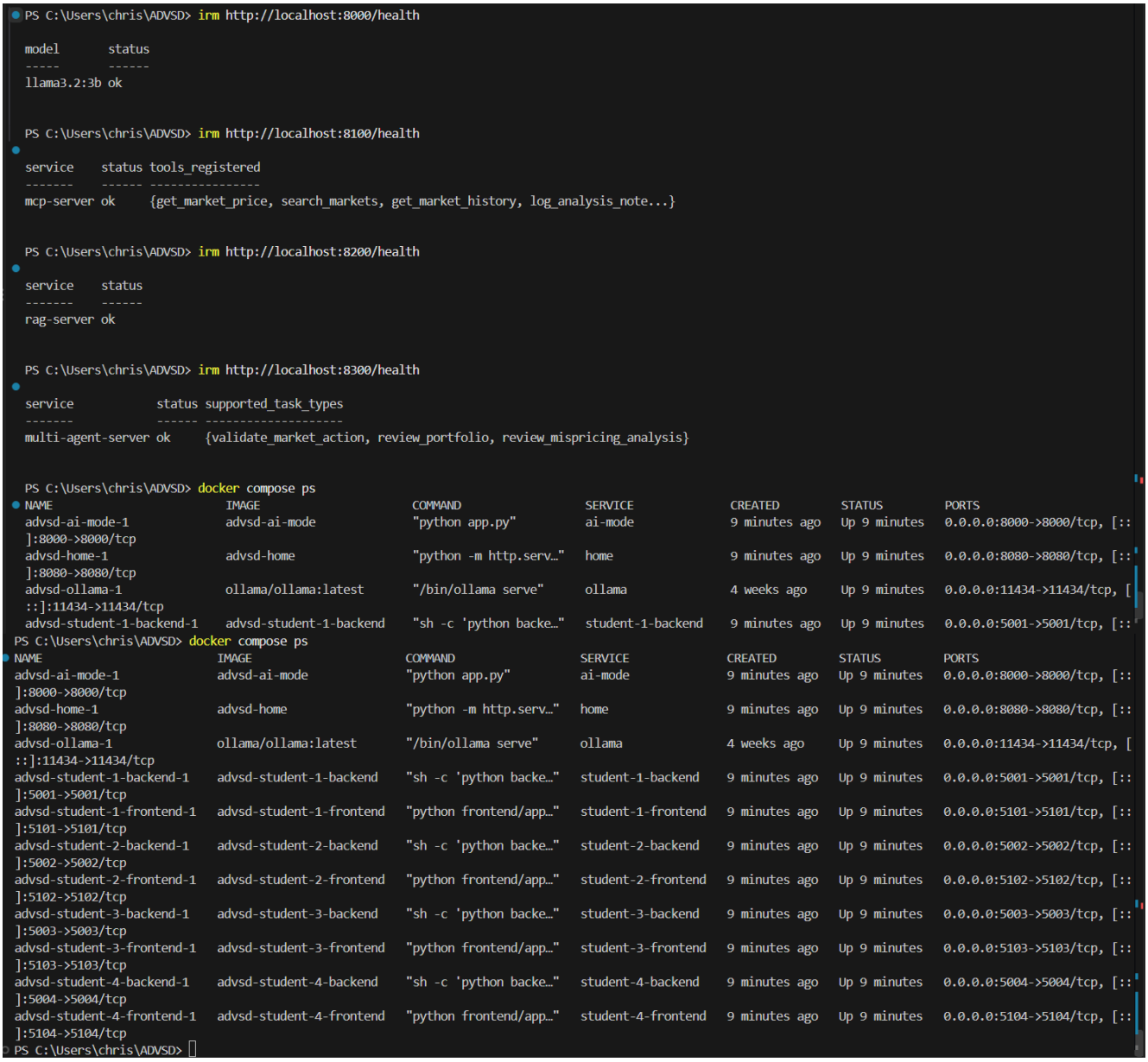
- All scans clean and all tests passing; no action needed.

The above is Validation log: `docs/release-2/review-logs/testing-review_20261028_235845.json`

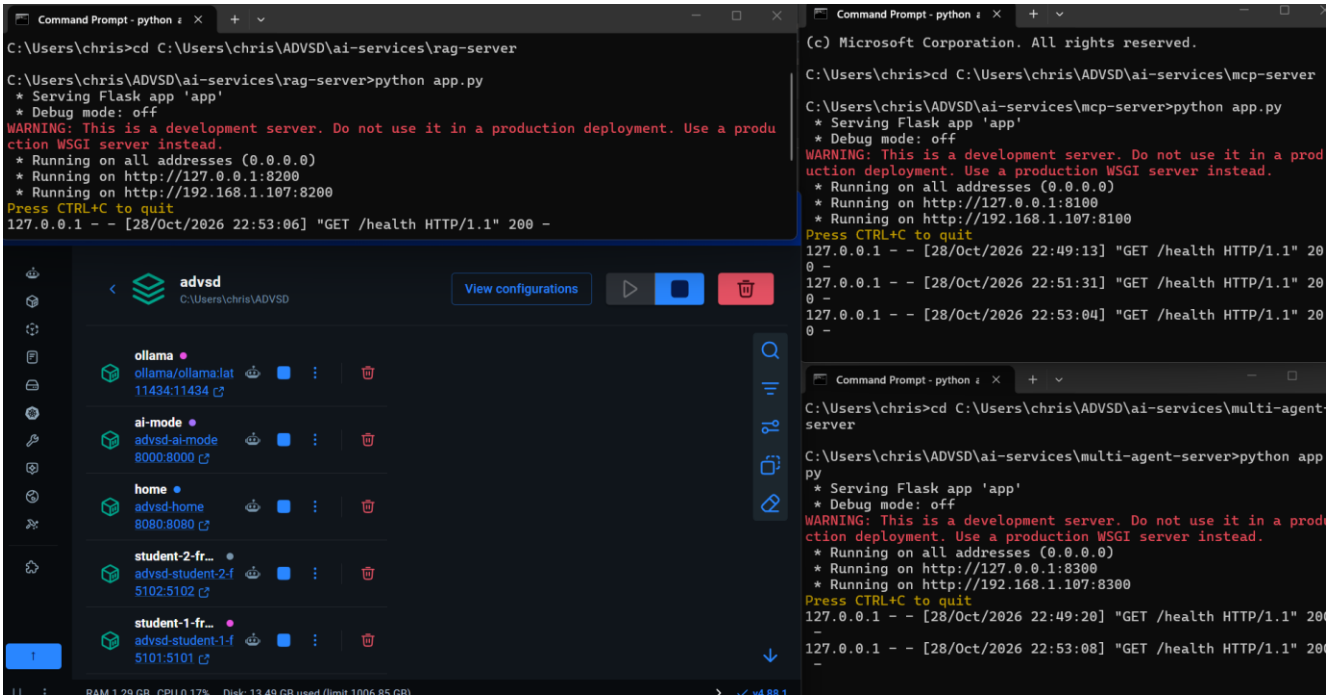
The review ran Ruff (security rules), detect-secrets and pytest for all four students and read each student's latest `student-x.yml` run on `main` from GitHub Actions. pip-audit was skipped here with `--skip-audit` because it is run and evidenced separately per student in Section 5.2. Overall risk: **Low**. All scans were clean and all tests passed, so no action was needed.

8. Local Deployment Evidence

The integrated Release 2 application runs locally with the frontend, backend/API and database microservices containerised through Docker Compose, while AI-Mode, MCP, RAG, the Multi-Agent Server and the shared agentic loop remain non-containerised local services, each started in its own terminal and reachable from the frontend UI through the backend/API.



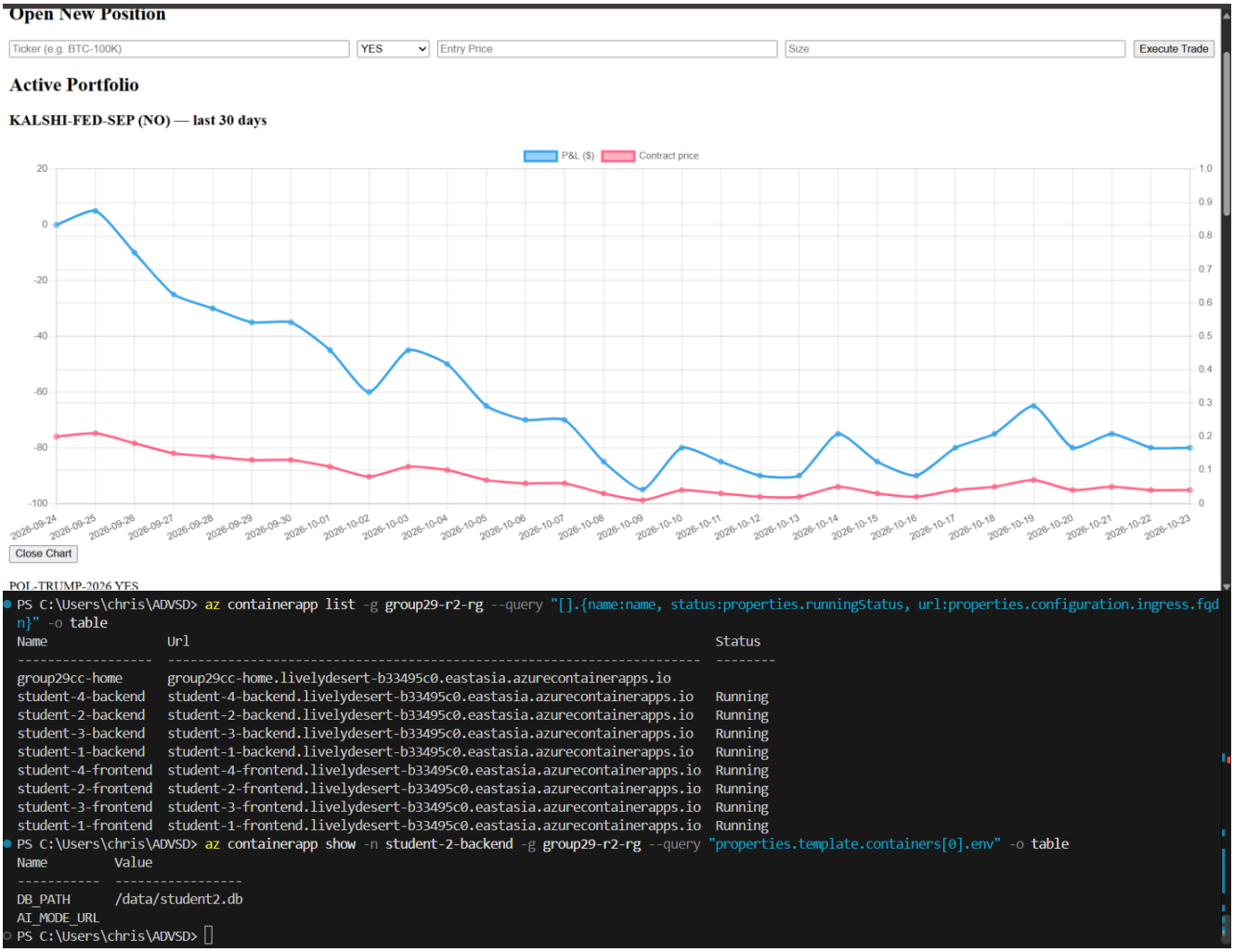
The student frontend, backend and database services run in Docker Compose; AI-Mode, MCP, RAG and the Multi-Agent Server each run as a separate non-containerized process. For example, A Multi-Agent Portfolio Review triggered from the student 2 UI passes through the containerized backend to the multi-Agent Server on :8300, which uses MCP on :8100 for live evidence, and returns the run for human review.



9. Cloud Deployment Report and Review

9.1 Cloud Deployment Report

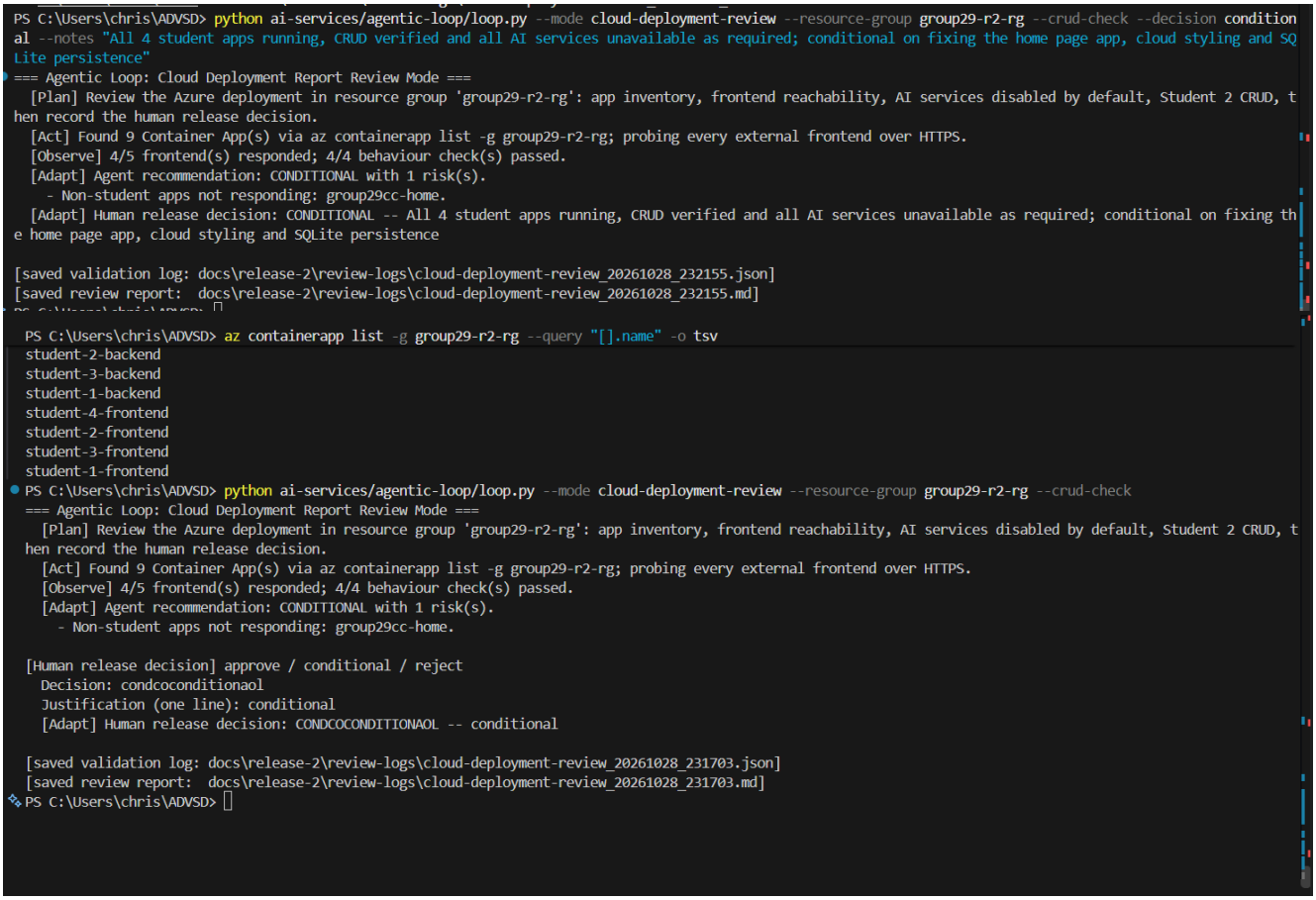
```
PS C:\Users\chris\ADVSD> az resource list -g group29-r2-rg --query "[].{name:name, type:type}" -o table
Name
------------------------
group29cc-logs
group29cc-env
group29ccstbyvfy5nt5l6bc
group29ccacr
group29cc-home
student-2-backend
student-4-backend
student-3-backend
student-1-backend
student-4-frontend
student-3-frontend
student-2-frontend
student-1-frontend
PS C:\Users\chris\ADVSD> az containerapp list -g group29-r2-rg --query "[].{name:name, status:properties.runningStatus, url:properties.configuration.ingress.fqdn}" -o table
Name              Url                                                                  Status
----------------  -------------------------------------------------------------------  --------
group29cc-home    group29cc-home.livelydesert-b33495c0.eastasia.azurecontainerapps.io
student-4-backend  student-4-backend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-2-backend  student-2-backend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-3-backend  student-3-backend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-1-backend  student-1-backend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-4-frontend  student-4-frontend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-2-frontend  student-2-frontend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-3-frontend  student-3-frontend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
student-1-frontend  student-1-frontend.livelydesert-b33495c0.eastasia.azurecontainerapps.io  Running
PS C:\Users\chris\ADVSD>
```



The application was deployed to Microsoft Azure Container Apps in East Asia (Azure for Students blocks Australia East) using the shared Bicep template (scripts/deploy/azure/main.bicep and modules/student-service.bicep) and scripts/deploy/azure/deploy.sh. The script creates the resource group, then runs a two-pass deployment. The first pass creates the Azure Container Registry, the Log Analytics workspace, the Container Apps environment and the apps; the student and home images are then built with docker build and pushed to the registry; the second pass points every app at its real image. The result is nine Container Apps: the shared home page, plus a frontend and a backend for each of the four students, with HTTPS ingress on each frontend. enableCloudAi defaults to false, so neither Ollama nor AI-Mode is deployed. MCP, RAG and the Multi-Agent Server are local-only services with no cloud URL, so every AI route returns “unavailable” while CRUD keeps working. The cloud-deployment.yml workflow deploys the same template only after all four student CI workflows on main pass. SQLite lives in each container, so data resets restart (Azure Files isn’t supported in Express environments; see Section 15).

Public URL: https://student-2-frontend.livelydesert-b33495c0.eastasia.azurecontainerapps.io/

9.2 Cloud Deployment Review (Agentic-Loop Cloud Deployment Review Mode)



Human release decision: Conditional. The baseline deployment works (all apps running, CRUD verified, AI off by default), but it isn't production-ready until the theme loads from the cloud rather than localhost and data persists across restarts.

10. Multi-Agent Workflow Evidence

This section evidences a complete Planner → Worker → Reviewer → Human Review cycle for at least one real task in the application, following the same four-stage structure as the Release 0/1 agentic loop.

Validate a Market Research Note against live market data. Note 43 (the chipmaker article) claimed a 35% probability for market 1426260 and the task was to check that claim against the live price.

Plan	Planner Agent	Task `validate_market_action` for the `research-notes` feature. Steps: (1) fetch the current live price for market 1426260 via the shared MCP server; (2) compare the submitted value (0.35) against the live probability and flag any gap worth a human's attention. Required evidence: a live market snapshot from the MCP `get_market_price` tool, and the numeric delta between the submitted value and the live probability.
Act	Worker Agent	Called MCP `get_market_price` for market 1426260 (source: Polymarket) using the submitted value supplied by the application. The live probability returned was 0.175. It computed the delta between the submitted 0.35 and the live 0.175 as 0.175. The MCP server was reachable.
Observe	Reviewer Agent	Both required pieces of evidence were gathered (evidence satisfied). Risk flagged: the submitted value drifted 17.50% from the live market price (threshold 5%), so the value may be stale or mistyped. Recommendation: approve with caution. Confidence: Medium.
Adapt	Human Review	Approved (run 8b81c2d6466d, decided 2026-10-23 00:29) Approved with caution: the live price is 17.5% against the notes 35% so the notes claim needs revisiting.

[CLAIM: Workflow history / coordination log]


```
{
  "run_id": "8b81c2d6466d",
  "task_type": "validate_market_action",
  "params": {
    "feature": "research-notes",
    "market_id": "1426260",
    "source": "both",
    "submitted_value": 0.35
  },
  "plan": {
    "task_type": "validate_market_action",
    "feature": "research-notes",
    "steps": [
      "Fetch the current live price/probability for market '1426260' (source=both) via the shared MCP server, on behalf of the 'research-notes' feature.",
      "Compare the submitted value (0.35) against the live probability and flag any gap worth a human's attention."
    ],
    "required_evidence": [
      "Live market snapshot for '1426260' from the MCP server's get_market_price tool.",
      "Numeric delta between submitted_value and the live probability."
    ]
  },
  "worker_output": {
    "evidence_gathered": [
      "MCP get_market_price returned probability=0.175 for '1426260' (source=polymarket).",
      "Computed delta between submitted_value (0.35) and live probability (0.175): 0.175."
    ],
    "mcp_reachable": true,
    "live_market": {
      "category": "General",
      "close_date": "2027-01-01T04:59:00Z",
      "id": "1426260",
      "probability": 0.175,
      "source": "polymarket",
      "title": "Will AI-chip export licensing become law this year?",
      "volume": 6265.805798,
      "yes_ask": 0.23,
      "yes_bid": 0.12
    },
    "delta": 0.175,
    "notes": []
  },
  "review": {
    "risks": [
      "Submitted value drifted 17.50% from the live market price (threshold 5%) -- value may be stale or mistyped."
    ],
    "recommendation": "approve_with_caution",
    "confidence": "Medium",
    "evidence_satisfied": true
  },
  "status": "approved",
  "human_decision": {
    "decision": "approve",
    "notes": null,
    "corrected_output": null
  },
  "created_at": "2026-10-23T00:28:45",
  "decided_at": "2026-10-23T00:29:11"
}
```


11. GitHub Commit Logs

Repository URL: https://github.com/Erfanur1/ADVSD

Each student's Release 2 contributions are identifiable in the commit history by GitHub author:

- Student 1 – Alex Choi: https://github.com/Erfanur1/ADVSD/commits/main/?author=AlexChoi2004
- Student 2 – Christopher Kumarelil Cherian: https://github.com/Erfanur1/ADVSD/commits/main/?author=christopherkumarelilcherian-bit
- Student 3 – Syed Humza Hussain: https://github.com/Erfanur1/ADVSD/commits/main/?author=SHumzaHussain
- Student 4 – Erfanur Rahman: https://github.com/Erfanur1/ADVSD/commits/main/?author=Erfanur1

Screenshots of each workflow's run history are in Section 6.2 (GitHub Actions Workflow Evidence); this section covers commit attribution only.

12. Individual Contribution Log

Each row records one student's Release 2 contribution: the code and report work they completed, how they validated it, and their approximate commit count for this release.

Student	Release 2 Work Completed	Integration / Validation Activities	GitHub Commits (approx.)
Student 1 Alex Choi	CODE: shared Multi-Agent Server (Planner/Worker/Reviewer + human review), shared pre-commit config, Azure Bicep IaC and deploy scripts, cloud-deployment.yml (shared tasks 1–4); Student 1 Multi-Agent integration, UI	Ran Multi-Agent validate and review end to end with a human decision; ran pre-commit scans on every commit; verified student-1-ci passes on PR and on main;	15 (+3 PR merges)

Student	Release 2 Work Completed	Integration / Validation Activities	GitHub Commits (approx.)
	panel, 14 endpoint tests, AI-assisted test job. REPORT: Sections 5.1, 6 (Student 1).	verified CRUD, AI-Mode, MCP and RAG routes locally.	
Student 2 Christopher Kumarelil Cherian	CODE: review_portfolio task (Planner, Worker, Reviewer) and the --portfolio-id CLI option; Student 2 Multi-Agent routes and panel; 16 endpoint tests and the pytest hook; dependency and Ruff fixes; agentic-loop review modes (multi-agent-review, testing-review, cloud-deployment-review) with 12 offline tests. Ran the Azure deployment from his own machine. REPORT: Sections 1, 3, 6 (Student 2), 7, 8, 9, 10, 12 (Student 2) and 15.	Ran Planner → Worker → Reviewer → Human Review from the UI and the CLI; ran all three review modes and saved their logs; verified student-2-ci on PR #21 and main; verified CRUD and the disabled-AI behaviour on the Azure deployment.	3 (+2 PR merges)
Student 3 Syed Humza Hussain	CODE: Integrated Market Research Notes and News Feed with the Multi-Agent Server: POST /multi-agent/validate and POST /multi-agent/review, a "Fact-Check a note" panel with the human decision buttons and a graceful fallback. Fixed Ruff finding (4xS110 resolved, 2xS104 documented) upgraded vulnerable dependencies and added a student 3 pytest hook. Wrote 20 endpoint tests fixed a 500 error in POST /notes and added a post-commit AI-assissted unit testing job to student-3.yml. Added a Multi-Agent-URL to docker-compose.yml. REPORT: Sections 1.3, 5.2, 6.1 and 6.2 Section 12 and Section 15 (student 3 items)	Ran the fact-check end to end against the real Multi-Agent server with live polymarket data (note 43 against market 1426260: Live price 20% claimed 35% drift flagged human approved) Ran pre-commit scans and the 26 tests locally before pushing. Verified student-3-ci passes and the pull request	1 commit PR#22
Student 4 Erfanur Rahman	CODE: review_mispricing_analysis task and Student 4 Multi-Agent integration; cloud deployment fixes (AI disabled by default, OIDC auth, verify-CI gate, deploy.sh, Azure Files removal, API version); pytest-student-4 hook. REPORT: Section 4 *(confirm)*. REPORT: Sections 1.3, 5.2, 6.1 and 6.2	Verified the Multi-Agent run live against the real MCP and Multi-Agent servers (HTTP and CLI); ran the cloud-deployment workflow and fixed each blocker it hit; verified student-4-ci on PR #34 and on main.	12 (+12 PR merges)

13. Bonus Evidence (Optional)

Bonus marks are awarded only after the required baseline cloud deployment (Section 9) is operational, with AI-Mode, MCP, RAG and the Multi-Agent Server disabled by default. Mark each row as attempted or not; only attempted rows need evidence.

#	Bonus Criterion	Attempted?	Evidence
B1	Cloud AI-Mode (+1): enabled and working in the cloud, with a successful interaction through the app.	*[Yes / No]*	*[evidence or N/A]*

#	Bonus Criterion	Attempted?	Evidence
B2	Cloud MCP (+1): successful MCP tool call through the deployed application.	*[Yes / No]*	*[evidence or N/A]*
B3	Cloud RAG (+1): grounded RAG answer with citations, generated in the cloud.	*[Yes / No]*	*[evidence or N/A]*
B4	Cloud Multi-Agent (+1): full Planner → Worker → Reviewer run in the cloud.	*[Yes / No]*	*[evidence or N/A]*
B5	Cloud Endpoint Security (+1): controls protecting exposed frontend/API endpoints.	*[Yes / No]*	*[evidence or N/A]*
B6	Cloud Data Security (+1): controls protecting application data, secrets and credentials.	*[Yes / No]*	*[evidence or N/A]*

14. Showcase Video URL

Maximum 10 minutes. Must show: the Multi-Agent workflow through the frontend UI, the frontend UI in both local and cloud deployments, pre-commit security scans, post-commit CI/CD endpoint testing, and the agentic-loop review modes. All students must participate and be able to explain and defend their own Release 2 contribution.

[CLAIM: Video link]

https://youtu.be/a_PK-YPBnRw

15. Known Issues and Limitations

- Carried over from Release 1: docker-compose.yml previously still defined ollama and ai-mode as Compose services, which contradicts the non-containerised requirement for these services. This must be fixed before Release 2 is submitted – confirm it has been resolved and update/remove this bullet once verified.
- Student-3: fact check needs the user to type a live marketID because notes are linked to news articles rather than markets. It uses the shared validate_market_action task so the check is meaningful but the link between a note and marker is manual. The post-commit AI-assissted test job skips in GitHub Actions because no AI-Mode is available there. Suggestions are generated oly when it is run locally with AI-Mode up.